

Aufgabe 1: Grundlagen (15 Punkte)

Was geben die folgenden „Python 3“-Programme aus?

(a) (2 Punkte)

```
1 pi = 3.141592653
2 e  = 2.718281828
3 print(pi <= e)
```

False

(b) (2 Punkte)

```
1 l = ["010", 5, 7, 2]
2 for i in range(len(l)):
3     print(int(i)+int(i))
```

0

2

4

6

(c) (2 Punkte)

```
1 l = ["010", 5, 7, 2]
2 for i in range(len(l)):
3     print(str(i)+str(i))
```

00

11

22

33

(d) (3 Punkte)

```
1 i = 42
2 if str(i) > "100":
3     print("größer")
4 if int(i) < 100:
5     print("kleiner")
6 if float(i) == 100.0:
7     print("gleich")
```

größer

kleiner

(e) (3 Punkte)

```
1 a = "Viel Glück in dieser Klausur"
2 print(a[14], a[23], a[12], a[9], a[18])
```

d a n k e

(f) (3 Punkte)

```
1 dieAntwort = 42*42+42/42 > 5
2 if dieAntwort:
3     print("Ja")
4 else:
5     print("Nein")
```

Ja

Aufgabe 2: Wohnen und Coden..... (30 Punkte)

In dieser Aufgabe geht es um die Berechnung von Wohnflächen. Zur Erinnerung: die Fläche eines rechteckigen Zimmers lässt sich als Produkt der Länge und Breite berechnen. Sie dürfen in dieser Aufgabe davon ausgehen, dass alle Zimmer rechteckig sind.

- (a) (16 Punkte) Schreiben Sie eine „Python 3“-Funktion `readData(filename)`. Die Funktion soll die übergebene Textdatei einlesen, daraus eine `list` erstellen und diese `list` zurückgeben. Die Textdatei enthält in jeder Zeile drei Informationen: den Namen des Zimmers, die Länge, sowie die Breite, diese sind jeweils separiert durch Leerzeichen. Hier ein Beispiel:

```
Wohnzimmer 4 6.2
Bad 2.1 4
Schlafzimmer 4 4.2
```

Die Größenangaben (Länge, Breite) sind vom Typ `float`. Ihre `list` soll pro Zimmer jeweils wiederum eine `list` mit zwei Einträgen enthalten, dem Zimmernamen und der Fläche dieses Zimmers. Für das obige Beispiel muss Ihr Ergebnis so aussehen: `[['Wohnzimmer', 24.8], ['Bad', 8.4], ['Schlafzimmer', 16.8]]`. Sollte die Datei nicht existieren oder nicht gelesen werden können, muss Ihre Funktion eine leere `list` zurückgeben.

Tip: Um eine Zeile in einzelne Elemente aufzubrechen, können Sie auch die Funktion `split` verwenden. Beispiel:

```
line = "Hallo Welt 2"
line.split() # ergibt: ['Hallo', 'Welt', '2']
```

```
1 def readData(filename):
2     try:
3         with open(filename, "r") as source:
4             result = []
5             lines = source.readlines()
6             for line in lines:
7                 tmp = line.split()
8                 name = tmp[0]
9                 area = float(tmp[1]) * float(tmp[2])
10                room = [name, area]
11                result.append(room)
12
13            return result
14    except:
15        return []
```

- (b) (12 Punkte) Schreiben Sie eine „Python 3“-Funktion `largestRoom(rooms)`, die eine `list` als Eingabe bekommt. Die `list` ist so aufgebaut, wie es für das Ergebnis der ersten Teilaufgabe gefordert war, also beispielsweise `[['Wohnzimmer', 24.8], ['Bad', 8.4], ['Schlafzimmer', 16.8]]`. Ihre Funktion soll nun den Raum mit der größten Fläche bestimmen und den dazugehörigen Zimmernamen zurückgeben. Sie dürfen davon ausgehen, dass alle Räume eine unterschiedliche Fläche besitzen.

```
1 def largestRoom(rooms):
2     winnerName = ""
3     winnerArea = 0
4     for room in rooms:
5         if winnerArea < room[1]:
6             winnerName = room[0]
7             winnerArea = room[1]
8     return winnerName
```

- (c) (2 Punkte) Schreiben Sie ein einzeliges „Python 3“-Hauptprogramm, welches die Lösungen der Aufgabenteile a) und b) verwendet, um den größten Raum aus der Datei `myHouse.txt` zu ermitteln und diesen auf der Standardausgabe anzuzeigen.

```
1 print(largestRoom(readData("myHouse.txt")))
```

Aufgabe 3: Semantische Fehlersuche.....(16 Punkte)

Die folgenden „Python 3“-Funktionen beinhalten Fehler, welche für bestimmte Eingaben nicht das vorgegebene Ergebnis liefern. Beschreiben Sie möglichst präzise, für **welche Eingaben** das Programm nicht die korrekte Lösung berechnet.

- (a) (6 Punkte) Die Funktion `listMin` soll das Minimum der Elemente der `list a`, welche nur Zahlenwerte beinhaltet, bestimmen und zurückgeben.

```
1 def listMin(a):
2     minValue = 0
3     for v in a:
4         if v < minValue:
5             minValue = v
6     return minValue
```

Wenn **a** keine Werte kleiner oder gleich 0 beinhaltet.

- (b) (10 Punkte) Die Funktion `listSum` soll die Summe der Elemente der `list a`, welche nur Zahlenwerte beinhaltet, bestimmen und zurückgeben.

```
1 def listSumRek(a,s,e):
2     if s == e:
3         return a[s]
4     else:
5         m = (s+e)//2
6         return listSumRek(a,s,m) + listSumRek(a,m+1,e)
7
8 def listSum(a):
9     return listSumRek(a,0,len(a)-1)
```

Wenn **a** leer ist.

Aufgabe 4: Quersumme (25 Punkte)

- (a) (8 Punkte) Schreiben Sie eine „Python 3“-Funktion `quer(zahl)`, welche die Quersumme einer Zahl berechnet. Die Quersumme einer Zahl ist definiert als die Summe der einzelnen Ziffern. Beispielweise ist die Quersumme von 32 der Wert $5 = 3 + 2$ und für die Zahl 10202 ist es $5 = 1 + 0 + 2 + 0 + 2$.

```

1 def quer(zahl):
2     result = 0
3     while zahl > 0:
4         result += zahl % 10
5         zahl //= 10
6     return result

```

- (b) (5 Punkte) Schreiben Sie eine „Python 3“-Funktion `querArray(arr)`, welche die Summe aller Quersummen eines Arrays innerhalb linearer Laufzeit berechnet zurückgibt. Beispielweise soll `querArray([5, 11, 23, 47])` das Ergebnis 23 liefern. Sie dürfen die Funktion aus der ersten Teilaufgabe hier verwenden.

```

1 def querArr(arr):
2     result = 0
3     for a in arr:
4         result += quer(a)
5     return result

```

- (c) (12 Punkte) Schreiben Sie eine „Python 3“-Funktion `checkQuerElement(arr)`, welche ein Array mit Zahlen als Eingabe bekommt und zunächst mittels der o.g. `querArray`-Funktion die Summe aller Quersummen berechnet. Dann soll die Funktion **schneller als in linearer Zeit** ermitteln, ob sich diese Summe selbst wieder im Array finden lässt. Das Ergebnis dieser Berechnung soll zurückgegeben werden. Sie dürfen davon ausgehen, dass die Zahlen im Array in aufsteigender Reihenfolge gespeichert sind. Beispielweise soll `checkQuerElement([5, 11, 23, 47])` das Ergebnis *True* liefern.

```

1 def binary_search(arr, low, high, x):
2     if high >= low:
3         mid = (high + low) // 2
4         if arr[mid] == x:
5             return mid
6         elif arr[mid] > x:
7             return binary_search(arr, low, mid - 1, x)
8         else:
9             return binary_search(arr, mid + 1, high, x)
10    else:
11        return -1
12
13 def checkQuerElement(arr):
14    return binary_search(arr, 0, len(arr)-1, querArr(arr)) >= 0

```

Aufgabe 5: Python vs. Java (14 Punkte)

Schreiben Sie folgendes „Python 3“-Programm in Java neu. Das Java-Programm soll in der Datei `Sublist.java` gespeichert werden. Hinweis: Der Operator für das logische und wird in Java `&&` geschrieben.

```
1 def isSublist(a, b):
2     if len(b) == 0:
3         return True
4     elif len(b) > len(a):
5         return False
6     else:
7         for i in range(len(a)):
8             if a[i] == b[0]:
9                 n = 1
10                while (n < len(b)) and (a[i+n] == b[n]):
11                    n += 1
12                if n == len(b):
13                    return True
14        return False
15
16 a = [2,4,3,5,7]
17 b = [4,3]
18 print(isSublist(a, b))
```

```
1 public class Sublist{
2     public static boolean isSublist(int[] a, int[] b){
3         if(b.length == 0){
4             return true;
5         } else if(b.length > a.length){
6             return false;
7         } else {
8             for(int i=0; i<a.length;i++){
9                 if(a[i] == b[0]){
10                    int n = 1;
11                    while (n < b.length && a[i+n] == b[n]) {
12                        n += 1;
13                    }
14                    if(n == b.length) {
15                        return true;
16                    }
17                }
18            }
19            return false;
20        }
21    }
22
23    public static void main(String[] args){
24        int[] a = {2,4,3,5,7};
25        int[] b = {4,3};
26        System.out.println(isSublist(a, b));
27    }
28 }
```